MCMP117 · Week 7 · Course Notes

Fundamentals of Operating Systems

Virtual Memory & File Systems

How can a 4 GB program run in 2 GB of RAM — and where do files really live?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain the idea of virtual memory and why a program can run with only part of it in RAM.
2. Describe demand paging, the valid–invalid bit and, step by step, how the OS handles a page fault.
3. Compute the effective access time with page faults and the largest fault rate that keeps a given slow-down.
4. Explain copy-on-write and its link to `fork()`.
5. Draw frame tables for FIFO, OPT and LRU page replacement, count page faults, and explain Belady's anomaly and the clock algorithm.
6. Explain thrashing and the working-set model.
7. Describe file attributes, file operations, the open-file tables, and sequential, direct and indexed access (with block arithmetic).
8. Compare single-level, two-level, tree and acyclic-graph directories; use absolute and relative paths; explain inodes, hard links and symbolic links.

Contents

1 From Week 6 to Week 7 1

2 Virtual Memory 1

3 Demand Paging 2

4 Page Replacement 7

5 Thrashing and the Working Set 11

6 The File Concept 12

7 Access Methods 14

8 Directory Structures 16

9 What Linux Shows: Inodes and Links 18

10 Summary 20

11 Glossary 20

12 Try It Yourself 21

13 Review Questions 21

14 Midterm Review: Weeks 1–7 22

15 Looking Ahead: Week 8 and Beyond 23

1 From Week 6 to Week 7

Last week the **MMU** translated logical addresses into physical ones, **paging** split memory into pages and frames, a **page table** recorded the mapping, and the **TLB** cached recent translations. All of this silently assumed that *every page of a running process is in RAM*. This week we drop that assumption: a page may also live on the disk and be brought into memory only when needed. This idea, **virtual memory**, lets a computer run more programs than its RAM could hold. In the second half we turn to the other big storage abstraction of an OS: the **file**.

2 Virtual Memory

2.1 The idea

Look at a real program and you will find that much of it is rarely used: code that handles unusual errors, menu options few people click, arrays declared with room for $10\,000$ entries of which 50 are used. Even code that *is* used is not needed all at the same time. Programs show **locality of reference**: during any short period, a program works with only a few of its pages (the current function, its local variables, the part of an array being processed).

Virtual memory

Virtual memory separates the memory seen by the program (the *virtual* or logical address space) from the physical memory of the machine. Only the pages that are currently needed are kept in RAM; the rest stay on a disk area called the **backing store** (or swap space). The virtual address space can therefore be much *larger* than physical memory.

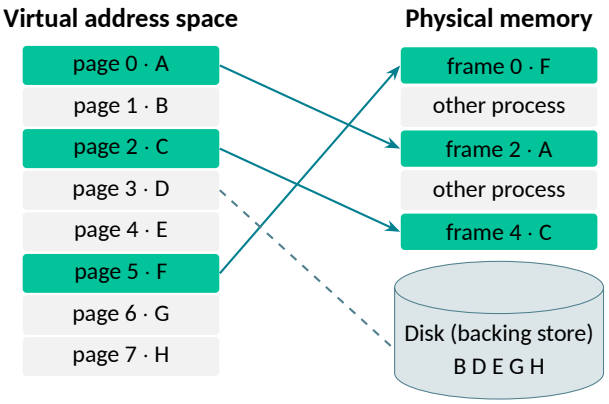


Figure 1: Only pages A, C and F are in RAM; the other pages of the program wait on the disk.

The benefits are considerable:

- A program is no longer limited by the size of physical memory. (On 64-bit Linux, each process has a virtual address space of 128 TiB.)
- Each process occupies less RAM, so more processes fit in memory at once: the *degree of multiprogramming* rises and with it CPU utilisation (Weeks 1 and 4).
- Less I/O is needed to load or swap a program, so programs start faster.
- Pages can be shared between processes (shared libraries, and copy-on-write after `fork()`, Section 3.5).

3 Demand Paging

3.1 Lazy loading

How does the OS decide which pages to put in RAM? The simplest strategy is to load the whole program before it starts. The alternative, used by every modern general-purpose OS, is **demand paging**: a page is loaded only when it is first referenced. This is an instance of a general programming idea called *lazy loading*: never do work before it is needed.

Compared with loading the whole program eagerly, demand paging needs less I/O and less memory, starts programs faster and lets more processes fit in RAM. The price is that the first use of each page causes a *page fault*.

The part of the OS that moves individual pages between disk and memory is called the **pager**. (The older word *swapper* refers to moving *whole processes*.) In *pure* demand paging a process starts with no pages in memory at all; its very first instruction causes a fault.

3.2 The valid--invalid bit

The hardware needs a way to tell whether a page is in memory. Each page-table entry has a **valid–invalid bit**:

- **v (valid):** the page belongs to the process *and* is in a frame; the entry holds the frame number.
- **i (invalid):** the page is either not in memory at the moment, or not part of the process's address

space at all.

Initially, every entry is set to i. While a process accesses only valid pages, it runs exactly as in Week 6, at full speed.

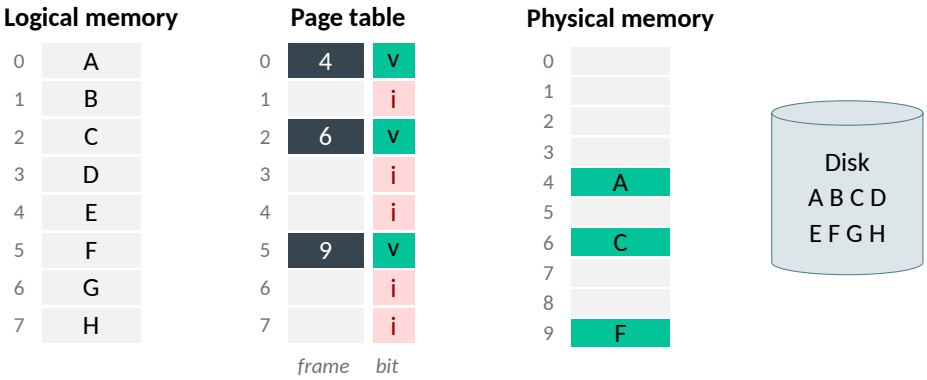


Figure 2: Pages A, C and F are in frames 4, 6 and 9 (bit v); all other entries are invalid. Every page has a copy on the disk.

3.3 Handling a page fault

What happens when the process touches a page whose bit is i? The MMU cannot translate the address, so it raises a **page fault**: a trap to the operating system (Week 2). The OS then works through the following steps.

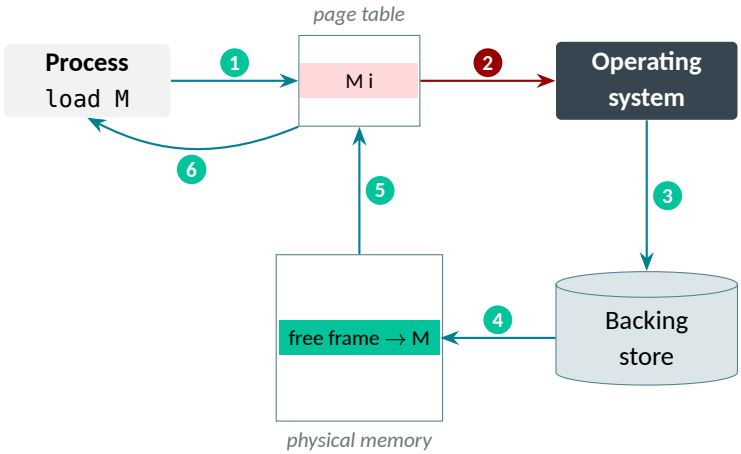


Figure 3: The six steps of page-fault handling.

1. **Reference.** The process executes an instruction that refers to page M. Its page-table entry says i.
2. **Trap.** The MMU raises a page fault. The OS saves the process state and checks an internal table: is the address legal at all? If not, the process is terminated (on Linux: *segmentation fault*). If it is legal but not in memory, continue.
3. **Locate.** The OS finds where page M is on the backing store and finds a free frame (if there is none, it must *replace* a page, Section 4).
4. **Read.** It schedules a disk read of the page into the frame. This takes milliseconds, so meanwhile the CPU scheduler (Week 4) runs another process; the faulting process is in the *waiting* state (Week 3).

5. **Update.** When the disk interrupt arrives, the OS writes the frame number into the page-table entry and sets the bit to v.
6. **Restart.** The process is made ready again, and the instruction that caused the fault is *restarted from the beginning*. This time the access succeeds; the process cannot tell that anything happened (except that it was slower).

Minor and major faults. Not every page fault needs the disk. If the page is brand-new memory (for example the first touch of a `malloc`'ed array), the OS only has to find a free frame and fill it with zeros. Linux calls this a **minor fault**. A **major fault** needs disk I/O, for example to read a page of a program file or a page that was swapped out. Major faults are thousands of times more expensive.

3.4 Effective access time with page faults

Let p be the **page-fault rate**, the probability that a memory access causes a page fault ($0 \le p \le 1$). With memory access time t_m and page-fault service time t_{fault}, the **effective access time** is the weighted average

$$\text{EAT} = (1 - p)\, t_m + p\, t_{\text{fault}}.$$

This has exactly the same structure as the TLB formula of Week 6: a fast case weighted by its probability plus a slow case weighted by its probability. The fault service time includes the trap, saving the state, the disk read (by far the largest part) and restarting the process.

EAT with page faults

Take $t_m = 200$ ns and $t_{\text{fault}} = 8$ ms $= 8\,000\,000$ ns, and suppose one access in a thousand faults, $p = 0.001$:

$$\begin{aligned} \text{EAT} &= 0.999 \times 200 \text{ ns} + 0.001 \times 8\,000\,000 \text{ ns} \\ &= 199.8 \text{ ns} + 8\,000 \text{ ns} = 8\,199.8 \text{ ns} \approx 8.2\ \mu\text{s}. \end{aligned}$$

Memory has become $8199.8/200 \approx 41$ times slower!

How small must p be for a slow-down below 10%? We need EAT $< 1.1\, t_m = 220$ ns:

$$200 + p\,(8\,000\,000 - 200) < 220 \quad \Longrightarrow \quad p < \frac{20}{7\,999\,800} \approx 2.5 \times 10^{-6}.$$

That is at most one page fault per $1/p \approx 400\,000$ memory accesses. (Numbers checked with `code/page_replacement.py`.)

The lesson: the page-fault rate must be extremely small, and it *is* small in practice only because of locality. With an SSD (t_{fault} of the order of 0.1 ms) the penalty is smaller, but the same reasoning applies.

3.5 Copy-on-write

In Week 3 we met `fork()`, which creates a child process as a copy of its parent. Copying every page of the parent would be slow, and often pointless: many children immediately call `exec()` and replace their whole memory with a new program. Virtual memory offers a better solution, **copy-on-write** (COW).

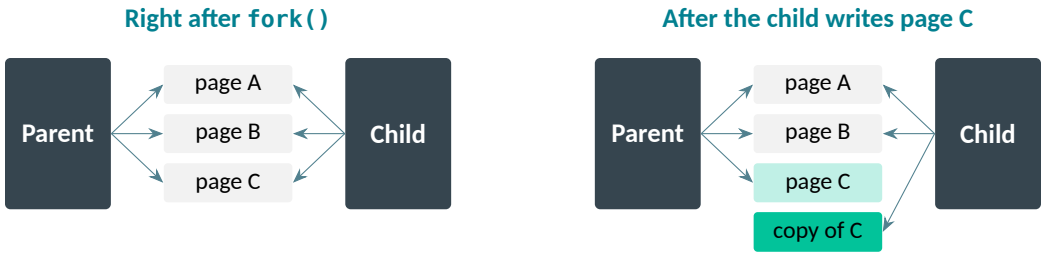


Figure 4: Copy-on-write: pages are shared until one process writes; only the written page is copied.

After `fork()`, parent and child share all their pages, and the OS marks them read-only. Reading is free. When either process *writes* to a shared page, the hardware raises a protection fault; the OS copies just that page, gives the writer its own copy (now writable) and restarts the write. Pages that are never written are never copied. The mechanism is the same trap-and-restart we used for demand paging.

3.6 Hands-on: watching page faults happen

The programs in this section are in the `code/` folder next to these notes. Compile everything with `make` (which uses `gcc -Wall -O2`).

Example 1: demand paging. The program allocates 64 MB with `malloc()`, then writes one byte into every page twice. The kernel counts page faults for each process; `getrusage()` reads the counter. The listing shows the core of the program; the complete version, which also prints the results, is `page_faults.c`.

```c
static long minor_faults(void)          /* kernel's counter */
{
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

int main(void)
{
    long page = sysconf(_SC_PAGESIZE);  /* 4096 bytes  */
    long size = 64 * MB;                /* 16384 pages */

    long f0 = minor_faults();
    char *a = malloc(size);             /* virtual only */
    long f1 = minor_faults();
    for (long i = 0; i < size; i += page)
        a[i] = 1;                       /* first touch  */
    long f2 = minor_faults();
    for (long i = 0; i < size; i += page)
        a[i] += 1;                      /* now resident */
    long f3 = minor_faults();
    /* print f1-f0, f2-f1, f3-f2 */
}
/* full program: code/page_faults.c */
```

Output on our test machine (Linux, 4 KB pages):

```
$ ./page_faults
page size 4096 bytes; array 64 MB = 16384 pages
after malloc       :      1 new faults
first touch (write):  16383 new faults
second touch       :      0 new faults
```

Reading the output:

- `malloc()` itself causes only 1 fault: it reserves *virtual* space and writes only a small bookkeeping header (into the first page).
- The first pass faults once for each page it touches: $16\,383$ faults, plus the 1 page already touched by `malloc` gives all $16\,384$ pages.
- The second pass causes 0 faults: every page is now resident.

These are all *minor* faults: the kernel only had to hand out zero-filled frames. The tool `/usr/bin/time -v` reports the counters for the whole run (including the faults for loading the program and the C library):

```
$ /usr/bin/time -v ./page_faults
...
   Maximum resident set size (kbytes): 66964
   Major (requiring I/O) page faults: 0
   Minor (reclaiming a frame) page faults: 16463
```

On systems where *transparent huge pages* are enabled for all memory, the kernel may map 2 MB pages instead and you will see far fewer faults. (Install the tool with `sudo apt install time` if `/usr/bin/time` is missing.)

Example 2: copy-on-write. The parent fills a 32 MB buffer and forks. The child first reads every page, then writes every page. The listing shows the core of the program; the complete version, with the same `minor_faults()` helper as Example 1, is `code/cow_fork.c`.

```
char *buf = malloc(SIZE);              /* 32 MB = 8192 pages */
memset(buf, 'A', SIZE);                /* parent owns them   */

pid_t pid = fork();
if (pid == 0) {                        /* ---- child ----    */
    long f0 = minor_faults();
    for (long i = 0; i < SIZE; i += page)
        sum += buf[i];                 /* READ: shared       */
    long f1 = minor_faults();
    for (long i = 0; i < SIZE; i += page)
        buf[i] = 'B';                  /* WRITE: copy page   */
    long f2 = minor_faults();
    /* print f1 - f0 and f2 - f1 */
    exit(0);
}
wait(NULL);
printf("parent still sees '%c'\n", buf[0]);
/* full program: code/cow_fork.c */
```

```
$ ./cow_fork
buffer: 8192 pages of 4096 bytes
child read  all pages:      0 faults
child write all pages:   8192 faults (copy-on-write)
parent still sees  'A'  (its copy was never changed)
```

Reading the shared pages costs nothing; writing costs exactly one fault per page, 8192 in total, because each page is copied on its first write. The parent still sees its original data. (The `fflush(stdout)` before `fork()` matters: without it, text still sitting in the parent's output buffer would be copied into the child and printed twice.)

4 Page Replacement

4.1 When memory is full

The OS deliberately *over-allocates* memory: the virtual memory of all processes together is larger than RAM, on the bet that not all pages are needed at once. Sooner or later a page fault occurs and there is no free frame. The OS must then choose a **victim** page to remove. The page-fault routine becomes:

1. Find the location of the wanted page on the disk.
2. Find a free frame. If there is none, use a **page-replacement algorithm** to select a victim frame; if the victim has been modified, write it to the disk; mark its page-table entry invalid.
3. Read the wanted page into the freed frame; update the page table.
4. Restart the process.

The modify (dirty) bit. Without care, a replacement costs *two* disk transfers: one to write the victim out and one to read the new page in. The hardware therefore keeps a **modify bit** (dirty bit) for every page, set automatically whenever the page is written. A *clean* victim, such as a page of program code, already has an identical copy on the disk and can simply be dropped.

How we compare algorithms. We run each algorithm on the same **reference string**, the sequence of page numbers that a program references, with the same number of frames, and count page faults. Throughout this section we use the classic string

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2,\ 1,\ 2,\ 0,\ 1,\ 7,\ 0,\ 1$$

with 3 frames, all initially empty. In the tables below, mint marks the page just loaded, pale columns are page faults, and grey columns are hits. All tables were produced by the Python simulator in the `code/` folder (`page_replacement.py`).

4.2 FIFO: first in, first out

The victim is the page that has been in memory the longest. FIFO is trivial to implement with a queue of page numbers.

reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
frame 0	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
frame 1		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
frame 2			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
fault?	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

Figure 5: FIFO with 3 frames: **15 page faults**.

The first three faults are **compulsory** (cold-start) faults: no algorithm can avoid loading 7, 0 and 1 the first time. Then 2 replaces 7 (the oldest), 0 is a hit, and 3 replaces 0, because 0 is now the oldest page — even though 0 is needed again at the very next reference. FIFO knows when a page arrived, not whether it is still being used.

4.3 OPT: the optimal algorithm

The victim is the page that *will not be used for the longest time* in the future. This algorithm (OPT or MIN, Bélády 1966) gives the lowest possible number of page faults for any reference string and number of frames.

reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
frame 0	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
frame 1		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
frame 2			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
fault?	F	F	F	F		F		F			F			F				F		

Figure 6: OPT with 3 frames: **9 page faults**, the minimum.

For example, at the reference to page 4 (position 8), the frames hold 2, 0 and 3. Looking ahead, 2 is used next at position 9, 3 at position 10 and 0 at position 11, so 0 is replaced. The catch is obvious: the OS cannot know the future reference string. OPT is used as a *yardstick*: "our algorithm is within 10% of optimal on this workload".

4.4 LRU: least recently used

LRU uses the recent past as an estimate of the near future: the victim is the page that has *not been used for the longest time*. Equivalently, it is OPT looking backwards in time.

reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
frame 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
frame 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
frame 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
fault?	F	F	F	F		F		F	F	F	F			F		F		F		

Figure 7: LRU with 3 frames: **12 page faults**.

At the reference to page 4, the frames again hold 2, 0 and 3, but now we look back: 2 was used longest ago, so 2 is replaced (OPT replaced 0). LRU works well because of locality.

Exact LRU needs hardware support on *every* memory access: either a time stamp written into the page-

table entry, or a stack of page numbers in which the referenced page is moved to the top. Both are too expensive for real hardware, so systems *approximate* LRU (Section 4.7).

4.5 Comparison

Algorithm	Victim is the page...	Faults (3 frames)	Implementable?	Belady's anomaly?
FIFO	loaded longest ago	15	yes, very cheap	possible
OPT	used farthest in the future	9	no (needs the future)	never
LRU	used longest ago	12	approximately	never

Exam tip: work column by column and write the frame contents after every reference. For OPT look *forward* in the string, for LRU look *backward*, for FIFO remember only the order of arrival.

4.6 Belady's anomaly

Intuition says that giving a process more frames can never increase its number of page faults. For FIFO, intuition is wrong. Consider the reference string $1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$:

reference	1	2	3	4	1	2	5	1	2	3	4	5
frame 0	1	1	1	4	4	4	5	5	5	5	5	5
frame 1		2	2	2	1	1	1	1	1	3	3	3
frame 2			3	3	3	2	2	2	2	2	4	4
fault?	F	F	F	F	F	F	F			F	F	

reference	1	2	3	4	1	2	5	1	2	3	4	5
frame 0	1	1	1	1	1	1	5	5	5	5	4	4
frame 1		2	2	2	2	2	2	1	1	1	1	5
frame 2			3	3	3	3	3	3	2	2	2	2
frame 3				4	4	4	4	4	4	3	3	3
fault?	F	F	F	F			F	F	F	F	F	F

Figure 8: FIFO on the same string: **9 faults with 3 frames** (top) but **10 faults with 4 frames** (bottom).

This is **Belady's anomaly**. LRU and OPT never show it. They belong to the class of **stack algorithms**: the set of pages in memory with n frames is always a subset of the set with $n + 1$ frames (for LRU: the n most recently used pages are always among the $n + 1$ most recently used). Adding a frame can therefore never add a fault. On the same string, LRU needs 10 faults with 3 frames and 8 with 4.

László Bélády

The Hungarian-born computer scientist László Bélády worked at IBM Research. In 1966 he published the optimal replacement algorithm (MIN), and in 1969, with R. A. Nelson and G. S. Shedler, the paper describing the anomaly that now carries his name.

4.7 Approximating LRU: reference bit and the clock

Many CPUs provide a **reference bit** for each page: the hardware sets it to 1 whenever the page is read or written. The OS can read it and clear it to 0. The bit does not say *when* a page was used, but it does say whether it was used *since the bit was last cleared*.

The **second-chance** or **clock** algorithm arranges the frames in a circle with a pointer, the "clock hand", at the next candidate:

- If the page under the hand has $R = 1$, it gets a second chance: set $R = 0$ and advance the hand.
- If it has $R = 0$, it has not been used since the hand last passed: it is the victim.

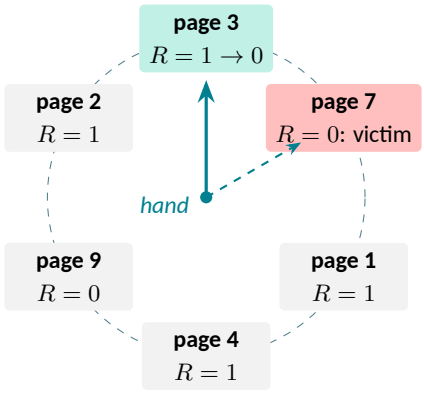


Figure 9: The hand is at page 3 ($R = 1$): its bit is cleared and the hand moves on to page 7 ($R = 0$), which is replaced.

If all bits are 1, the hand goes once around the circle clearing them and the algorithm degenerates to FIFO. Real systems use refinements of this idea; Linux, for instance, keeps "active" and "inactive" page lists.

5 Thrashing and the Working Set

5.1 Thrashing

Suppose a process is actively using more pages than it has frames. It faults, and the page it evicts is one it needs again soon, so it faults again, and again. A process that spends more time paging than executing is **thrashing**.

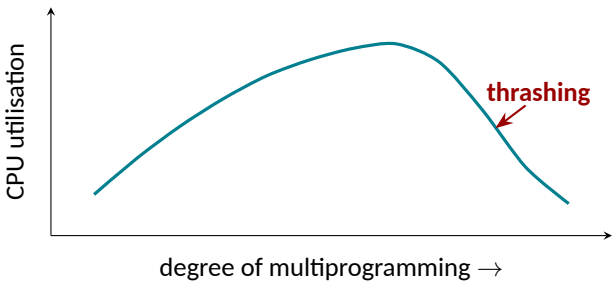


Figure 10: CPU utilisation against the number of processes in memory (conceptual curve, not measured data).

The curve explains the danger. Adding processes first raises CPU utilisation (more processes to run while others wait for I/O). Beyond some point the processes no longer have enough frames each; page faults

explode, processes queue for the paging disk, and CPU utilisation collapses. A naive scheduler that reacts to the idle CPU by admitting even more processes makes things worse — a vicious circle. The cure is to *reduce* the degree of multiprogramming (suspend or swap out some processes) or to add physical memory. If you have ever used a computer with too little RAM and too many browser tabs, with the disk light constantly on and the mouse barely moving, you have seen thrashing.

5.2 The working-set model

The **working set** of a process at time t, $\mathrm{WS}(t, \Delta)$, is the set of distinct pages referenced in its most recent Δ references; Δ is called the working-set window. The working set approximates the program's current locality.



Figure 11: The working set follows the program's locality: 5 pages at t_1, only 2 at t_2.

If $\mathrm{WSS}_i = |\mathrm{WS}_i|$ is the working-set size of process i, the total demand for frames is $D = \sum_i \mathrm{WSS}_i$. If D exceeds the number of available frames m, thrashing will occur, and the OS should suspend a process until frames become available. The choice of Δ matters: too small and it misses part of the locality; too large and it overlaps several localities and wastes frames.

A simpler practical relative is *page-fault frequency* control: the OS measures each process's fault rate, gives it more frames when the rate is too high, and takes frames away when it is very low.

6 The File Concept

Main memory is volatile and too small to keep everything. The OS therefore offers a second storage abstraction that survives power-off: the **file**, a named collection of related information recorded on secondary storage. From the user's point of view, a file is the smallest unit of storage that can be saved: you cannot write data to the disk except inside a file.

6.1 File attributes

Besides its contents, every file has **attributes** (metadata). The right-hand column shows the values for a real file, taken from the `stat` output in Section 9.1.

Attribute	Meaning	Example (`notes.txt`)
Name	Human-readable name; in Linux it is stored in the directory, not with the other attributes.	`notes.txt`
Identifier	Unique number of the file inside the file system.	inode 368662
Type	Regular file, directory, symbolic link, device, …	regular file
Location	Where the file's data blocks are on the device.	block pointers in the inode
Size	Current size in bytes (and blocks allocated).	6 bytes (8 blocks of 512 B)
Protection	Who may read, write, execute.	`-rw-rw-r--`, owner `student`
Timestamps	Creation, last access, last modification, last status change.	Access / Modify / Change / Birth

The OS keeps these attributes in a per-file structure called the **file control block** (FCB). In UNIX and Linux the FCB is the **inode** (Section 9).

6.2 File operations

A file is an *abstract data type*: it is defined by the operations you can apply to it. The basic operations, with the corresponding Linux system calls, are:

Operation	What the OS does	Linux call
create	Find space in the file system and add an entry to the directory.	`open(…, O_CREAT)`
open	Search the directory once, check permissions, return a *file descriptor*.	`open()`
read	Copy bytes from the file, starting at the current position.	`read()`
write	Copy bytes into the file, starting at the current position.	`write()`
seek	Move the current position ("reposition"); no data is transferred.	`lseek()`
close	Release the open-file table entry.	`close()`
delete	Remove the directory entry; free the space when no longer used.	`unlink()`
truncate	Erase the contents but keep the attributes.	`truncate()`

Other operations (append, rename, copy) can be built from these. Note the name `unlink()` for delete: it removes one *name* of the file; Section 8.3 explains why that is not always the same as deleting the data.

6.3 The open-file tables

Searching the directory and checking permissions for every single `read()` would be slow. Instead, `open()` does this work once and stores the result in memory tables; it returns a small integer, the **file descriptor**, which the process uses in all later calls. Descriptors 0, 1 and 2 are already open when a program starts: standard input, standard output and standard error (Week 2).

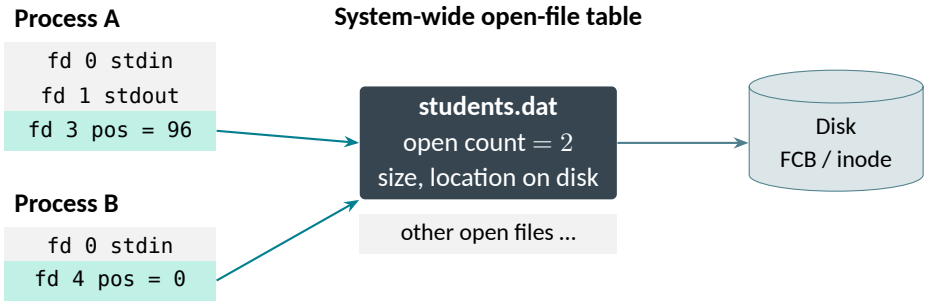


Figure 12: Per-process tables hold each process's file pointer; the system-wide table holds shared information and an open count.

- The **per-process open-file table** has one entry per open file of that process, with the **file pointer** (current position) and the access mode (read, write, ...).
- The **system-wide open-file table** has one entry per open file, shared by all processes that have it open, with process-independent information: location on the disk, size, and an **open count**. `close()` decrements the count; when it reaches 0 the entry is removed.

This is the textbook model. Linux adds one more level (descriptor $\rightarrow$ "open file description" holding the position $\rightarrow$ in-memory inode), but the idea is the same.

6.4 File types and extensions

The OS and applications must know what kind of data a file contains. The most common technique is to put the type into the name, as an **extension**:

File type	Usual extensions	Contents
Source code	`.c .py .java`	program text in a programming language
Object / executable	`.o .exe` (none on Linux)	machine code
Script	`.sh .bat`	commands for a command interpreter
Text / document	`.txt .pdf .docx`	text, formatted documents
Multimedia	`.jpg .mp3 .mp4`	image, sound, video
Archive	`.zip .tar.gz`	several files, often compressed

Windows uses the extension to decide which program opens a file. In UNIX and Linux an extension is only a *hint* for people and applications; the kernel does not care. The `file` command ignores the name and looks at the content, in particular at the first few bytes, which for many formats are a fixed **magic number** (an ELF executable starts with the bytes `7F 45 4C 46`, a JPEG image with `FF D8 FF`):

```
$ file file_access.c
file_access.c: C source, ASCII text
$ file file_access
file_access: ELF 64-bit LSB pie executable, x86-64, version 1...
$ file photo.txt
photo.txt: JPEG image data, JFIF standard 1.01, aspect ratio,...
$ file students.dat
students.dat: data
```

Here `photo.txt` is really a JPEG picture with a misleading name, and `students.dat` is the binary record file written by Example 4 (long lines are cut with `...`).

7 Access Methods

How can the information in a file be read? The OS offers a few standard **access methods**.

7.1 Sequential access

Information is processed in order, one record after the other. A *read next* returns the next portion of the file and advances the file pointer; *write next* appends and advances; *rewind* returns to the beginning. This is the model of a magnetic tape, and of most programs: editors, compilers and media players read their input from beginning to end.

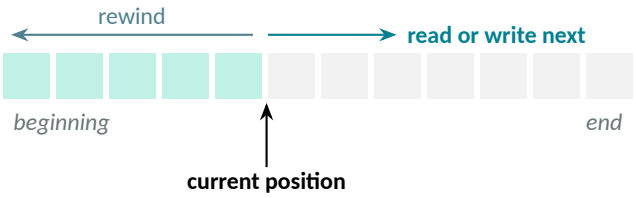


Figure 13: Sequential access.

7.2 Direct (relative) access

The file is viewed as a numbered sequence of fixed-size *blocks* or *records*, and any of them can be read or written in any order: *read* n, *write* n, or *seek* n followed by read next. This matches a disk, which can reach any block in roughly the same time. Databases and reservation systems need direct access: to find seat 17C on a flight, you do not want to read all the seats before it.

The block number n is *relative* to the beginning of the file: block 0, block 1, and so on. The OS translates relative block numbers into physical disk blocks; the user never needs to know where on the device the file lies. Sequential access can easily be simulated on a direct-access file by keeping a current position cp: *read next* means "read block cp, then $cp \leftarrow cp + 1$".

Where is record 1000?

A file holds student records of $R = 100$ bytes each, numbered from 0. The file system uses blocks of $B = 4096$ bytes. Which block holds record $k = 1000$, and where in the block does it start?

$$\begin{aligned} \text{byte offset} &= k \cdot R = 1000 \times 100 = 100\,000, \\ \text{block} &= \left\lfloor \frac{100\,000}{4096} \right\rfloor = 24, \\ \text{offset in block} &= 100\,000 - 24 \times 4096 = 100\,000 - 98\,304 = 1\,696. \end{aligned}$$

The record's last byte is $100\,099$, and $\lfloor 100\,099/4096 \rfloor = 24$ as well, so the whole record lies in relative block 24, bytes 1696 to 1795: one block read is enough. In C this is simply `lseek(fd, 1000 * 100, SEEK_SET)` followed by `read(fd, rec, 100)`; the OS performs the division.

7.3 Indexed access

For large files, even direct access needs to know *which* record number to read. An **index** solves this: a small, sorted file of (key, block number) pairs, for example student surnames with the block holding each student's record. To find a record, the OS or application searches the index (fast, because it is small and sorted) and then reads one block directly.

If the index itself becomes too large, it is indexed in turn (a multi-level index). This idea underlies database indexes such as B-trees, IBM's classic ISAM (indexed sequential access method), and the way file systems locate a file's own blocks.

7.4 Hands-on: sequential and direct access in C

Example 4 creates a file of five fixed-size 32-byte records, reads them sequentially, jumps directly to record 3 (byte $3 \times 32 = 96$) and overwrites record 1 in place.

The listing shows the core of the program; the complete version, `file_access.c`, also checks the return value of every call.

```c
#define RECSZ 32                        /* bytes per record */
int fd = open("students.dat", O_CREAT | O_RDWR | O_TRUNC, 0644);
/* ... write 5 records: "0 Ayse", "1 Mehmet", ... "4 Zeynep" */

lseek(fd, 0, SEEK_SET);                 /* 1) sequential   */
while (read(fd, rec, RECSZ) == RECSZ)
    printf("  [%s]", rec);

lseek(fd, 3 * RECSZ, SEEK_SET);         /* 2) direct: byte 96 */
read(fd, rec, RECSZ);

lseek(fd, 1 * RECSZ, SEEK_SET);         /* 3) overwrite rec 1 */
write(fd, rec, RECSZ);
close(fd);
/* full program: code/file_access.c */
```

```
$ ./file_access
sequential:  [0 Ayse]  [1 Mehmet]  [2 Elif]  [3 Can]  [4 Zeynep]
direct: record 3 at byte 96 -> [3 Can]
after update: record 1 -> [1 Mehmet (updated)]
file size: 160 bytes
```

The file pointer that `read()` advances and `lseek()` sets is exactly the "current position" kept in the per-process open-file table. The file is $5 \times 32 = 160$ bytes long; overwriting record 1 did not change its size or touch the other records.

8 Directory Structures

A disk can hold millions of files. A **directory** (folder) is a symbol table that maps file names to their FCBs. Operations on directories include searching for a file, creating and deleting files, listing a directory, renaming, and traversing the whole file system (for backups, for example). How directories are organised has evolved through several stages.

8.1 Single-level and two-level directories

In a **single-level directory**, all files of all users are in one directory. It is simple, but every name must be unique: if two users both want a file called `test`, one of them loses. With hundreds of files, a single list becomes unmanageable.

A **two-level directory** gives each user a private *user file directory* (UFD), listed in a *master file directory* (MFD). Two users can now both have a file called `test`; a file is named by a path such as `/ayse/test`. Users are isolated from one another — good for protection, but awkward when they want to share files, and there is still no way to group a user's own files.

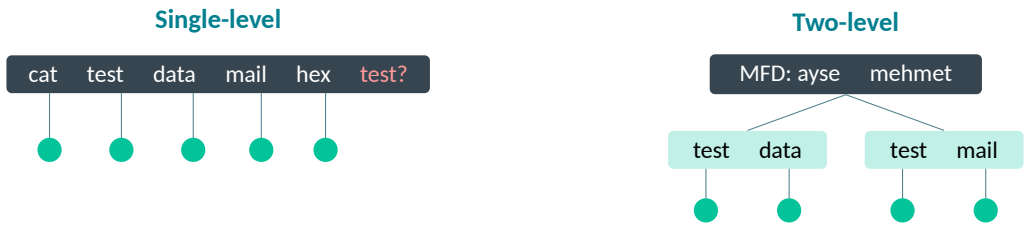


Figure 14: Single-level: one name space for everyone (the second `test` clashes). Two-level: one directory per user.

8.2 Tree-structured directories

The natural generalisation is a **tree**: a directory may contain files *and* other directories (subdirectories), to any depth. Every modern operating system uses a tree, or one of the generalisations below. Each process has a **current working directory**.

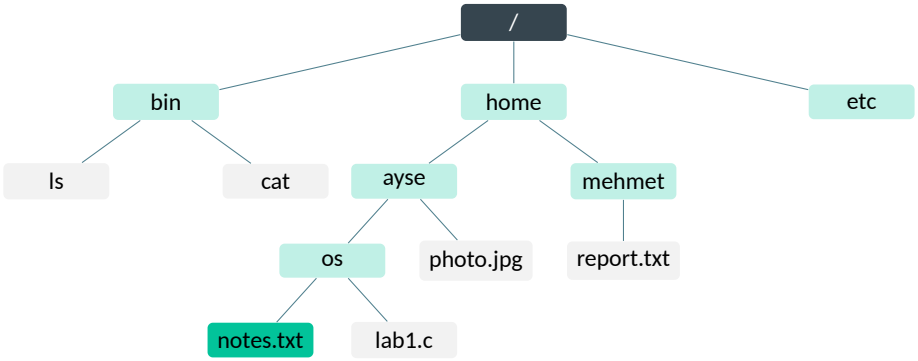


Figure 15: A tree-structured directory (pale: directories; grey: files).

A file can be named in two ways:

- `/home/ayse/os/notes.txt` is an **absolute path**: it starts at the root `/`, lists every directory on the way, and works from anywhere.
- A **relative path** starts at the current directory. If the current directory is `/home/ayse`, the same file is `os/notes.txt`.

Every directory contains two special entries: `.` (the directory itself) and `..` (its parent). From `/home/ayse/os`, Mehmet's report is `../../mehmet/report.txt`. In the shell, `~` is short for your home directory. Windows uses the same idea with drive letters and backslashes: `C:\Users\ayse\os\notes.txt`. In Week 9 you will move around this tree with `pwd`, `cd` and `ls`.

8.3 Acyclic-graph directories: hard and symbolic links

A tree cannot express *sharing*: if Ayşe and Mehmet work on the same project, the project file should appear in both their directories — not as two copies that drift apart, but as one file. An **acyclic-graph directory** allows a file or subdirectory to appear in several directories, as long as no cycles are formed. UNIX and Linux offer two kinds of **link**:

- A **hard link** is a second directory entry pointing to the *same* inode. Both names are equal; neither is "the original". The inode keeps a **link count**, the number of directory entries pointing to it. Deleting a name (`unlink`) decrements the count; the data is freed only when the count reaches 0 and no process has the file open. Hard links cannot cross file systems and (in Linux) cannot point to directories.
- A **symbolic link** (soft link) is a small separate file whose content is a *path name*. Opening it makes the OS follow the path. It can point to directories and across file systems, but if the target is deleted or moved, the link *dangles*.

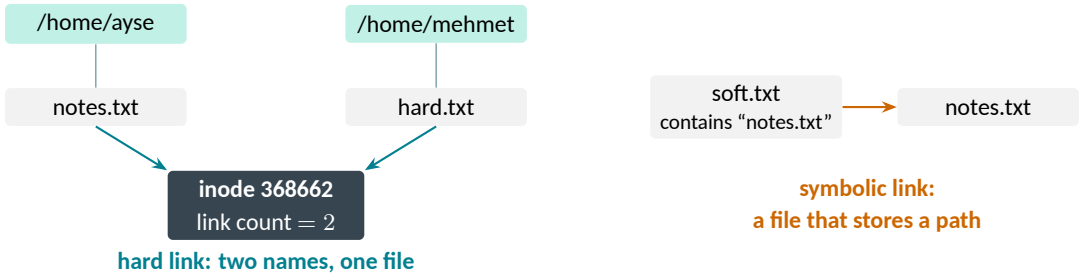


Figure 16: A hard link is another name for the same inode; a symbolic link is a file containing a path.

8.4 General-graph directories

If links are allowed to point to *any* directory, including an ancestor, the directory structure can contain **cycles**. Two problems appear: a program that walks the file system (search, backup) may loop forever, and a subtree that has been "deleted" may still be referenced from within itself, so its link count never drops to 0 and its space is never freed without expensive garbage collection. Practical systems avoid general graphs: Linux forbids hard links to directories, and when a chain of symbolic links loops, path lookup stops with the error EL00P ("too many levels of symbolic links").

9 What Linux Shows: Inodes and Links

In Linux, a directory is just a table of (name, inode number) pairs. Everything else about a file — type, permissions, owner, size, timestamps, link count and the pointers to its data blocks — is stored in its **inode**. The inode does *not* contain the file's name; that is why one inode can have several names.

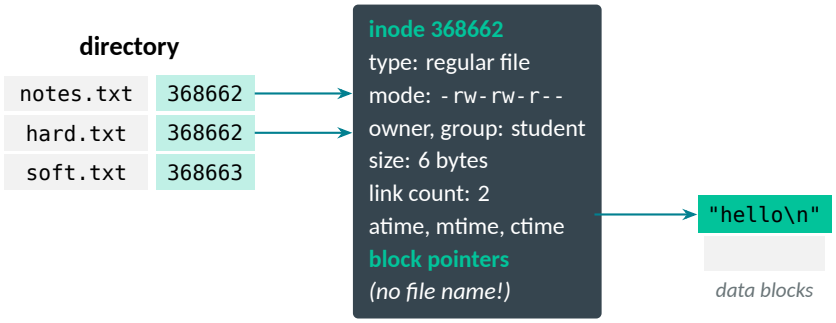


Figure 17: Directory entries point to inodes; inodes point to data blocks.

Useful commands: `ls -i` prints inode numbers, the second column of `ls -l` is the link count, `stat` shows the whole inode, and `df -i` shows how many inodes are free. A consequence worth knowing: `mv` within one file system only changes directory entries, so moving even a huge file is instant — the inode and data stay where they are.

9.1 Hands-on: `ls -li`, `ln`, `ln -s` and `stat`

The script `code/links_demo.sh` runs the following commands (better: type them one by one in an empty directory). Output captured as an ordinary user on our test machine:

```
$ echo "hello" > notes.txt
$ ls -li notes.txt
368662 -rw-rw-r-- 1 student student 6 Oct  8 12:20 notes.txt
$ ln notes.txt hard.txt
$ ln -s notes.txt soft.txt
$ ls -li
total 8
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 hard.txt
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 notes.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 368662      Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/ student)   Gid: ( 1002/ student)
Access: 2026-10-08 12:20:48.530146698 +0000
Modify: 2026-10-08 12:20:48.530146698 +0000
Change: 2026-10-08 12:20:48.534146698 +0000
 Birth: 2026-10-08 12:20:48.530146698 +0000
$ rm notes.txt
$ cat hard.txt
hello
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -li
total 4
368662 -rw-rw-r-- 1 student student 6 Oct  8 12:20 hard.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
```

What to observe:

- `notes.txt` and `hard.txt` have the *same* inode number (368662) and a link count of 2: two names, one file.
- `soft.txt` has its own inode (368663), type `l`, and size 9: exactly the length of the string `notes.txt` it stores.
- In `stat`, *Change* (ctime, the last change of the *inode*) is a few milliseconds later than *Modify* (mtime, the last change of the *data*): creating the hard link updated the link count in the inode, not the contents.
- After `rm notes.txt`, the data is still reachable through `hard.txt` (link count back to 1), while `soft.txt` dangles: the path it stores no longer exists.

This session is a preview of Week 10 (file operations: `cp`, `mv`, `rm`, `ln`) and Week 11, where the permission string `-rw-rw-r--`, stored in the inode, is explained in detail.

10 Summary

Week 7 in eight sentences

1. Virtual memory gives each process a large address space of which only the pages in use are in RAM; locality makes this work.
2. Demand paging loads a page on its first use; the valid–invalid bit tells the MMU whether a page is in memory.
3. An access to an invalid page traps to the OS (page fault), which loads the page and restarts the instruction; EAT $= (1-p)t_m + p\,t_{\text{fault}}$, so p must be tiny.
4. Copy-on-write shares pages after `fork()` until one process writes.
5. When no frame is free, a replacement algorithm picks a victim: FIFO (15 faults in our example, Belady's anomaly possible), OPT (9, optimal but needs the future), LRU (12, approximated by the clock algorithm).
6. Thrashing occurs when processes have fewer frames than their working sets; the cure is fewer processes in memory.
7. A file is a named collection of data with attributes; it is opened once and then accessed sequentially, directly (block arithmetic) or through an index.
8. Directories evolved from single-level to two-level to trees and acyclic graphs; in Linux, directory entries map names to inodes, and hard and symbolic links give a file several names.

11 Glossary

Backing store Disk area that holds the pages of processes that are not in RAM (swap space).

Belady's anomaly More frames causing more page faults; possible with FIFO.

Clock (second-chance) algorithm LRU approximation that scans frames in a circle using the reference bit.

Dirty (modify) bit Hardware bit set when a page is written; clean pages need not be written back.

File control block / inode Per-file structure holding a file's attributes and block locations.

Hard link An additional directory entry for an existing inode.

Page fault Trap raised when a process accesses a page that is not in memory.

Symbolic link A file whose content is the path of another file.

Thrashing State in which processes spend more time paging than executing.

Valid–invalid bit Page-table bit that tells whether a page is in memory.

Virtual memory Separation of a program's address space from physical memory, allowing it to be larger than RAM.

Working set Set of distinct pages referenced in the last Δ references.

12 Try It Yourself

Page faults, page replacement and links on your own machine

Linux (lab PCs, WSL, or a virtual machine), in the `code/` folder:

```
$ make                          # builds the three C programs
$ ./page_faults                 # change 64 to 256 MB: what happens?
$ /usr/bin/time -v ./cow_fork   # look for "Minor ... page faults"
$ python3 page_replacement.py   # edit refs and frames, rerun
$ mkdir lab && cd lab && bash ../links_demo.sh
```

1. In `page_faults.c`, touch only every *second* page in the first loop. How many faults do you expect, and how many do you get?
2. Change the reference string in `page_replacement.py` to one of your own and verify your hand-drawn FIFO, OPT and LRU tables.
3. Make a hard link to a file, edit the file through one name and look at it through the other. Then try `ln` on a directory: what error do you get?
4. Run `ls -li /` and `stat /`. Which inode number does the root directory have on your system?

13 Review Questions

Conceptual

1. What is virtual memory, and why can a program run when only some of its pages are in RAM?
2. Explain the difference between eager loading and demand paging. What is the price of demand paging?
3. What does the valid–invalid bit mean? Give two different reasons for a page to be marked i.
4. List the six steps of page-fault handling. Why must the faulting instruction be restarted?
5. Explain copy-on-write. Why does it make `fork()` followed by `exec()` cheap?
6. Why is OPT not implementable? Why is it still useful?
7. What is Belady's anomaly? Which of FIFO, OPT and LRU can suffer from it?
8. Describe thrashing and the vicious circle that can cause it. How does the working-set model help?
9. Compare sequential, direct and indexed access, with one application for each.
10. Compare hard and symbolic links: what happens to each when the original name is deleted?

Numerical

11. With $t_m = 200$ ns and $t_{\text{fault}} = 8$ ms, compute the EAT for $p = 0.0001$.
12. With $t_m = 100$ ns and $t_{\text{fault}} = 5$ ms, what is the largest p that keeps the slow-down below 20%?
13. For the reference string $1, 2, 3, 1, 4, 1, 2$ with 3 frames, count the page faults for FIFO, LRU and OPT.

14. For the reference string $7, 0, 1, 2, 0, 3, 0, 4$ with 4 frames, count the faults for FIFO and LRU.
15. Records are 64 bytes, blocks 4096 bytes. In which relative block, and at which offset, does record 200 start?
16. A file has 250-byte records and 1024-byte blocks. Does record 40 fit in a single block?

Answers to the numerical questions

11. EAT $= 0.9999 \times 200 + 0.0001 \times 8\,000\,000 = 199.98 + 800 = 999.98$ ns ≈ 1 μs, five times slower than RAM.
12. Need $100 + p\,(5\,000\,000 - 100) < 120$, so $p < 20/4\,999\,900 \approx 4.0 \times 10^{-6}$: at most one fault per $250\,000$ accesses.
13. FIFO: $1, 2, 3$ (3 faults), 1 hit, 4 replaces 1, 1 replaces 2, 2 replaces 3: 6 faults. LRU: 4 replaces 2 (least recently used), 1 hit, 2 replaces 3: 5 faults. OPT: 4 replaces 3 (never used again): 4 faults.
14. With 4 frames, $7, 0, 1, 2$ cause 4 compulsory faults, 0 is a hit, 3 is a fault and so is 4. FIFO: 3 replaces 7, 0 hit, 4 replaces 0: 6 faults. LRU: 3 replaces 7, 0 hit, 4 replaces 1: 6 faults.
15. Offset $200 \times 64 = 12\,800$; block $\lfloor 12\,800/4096 \rfloor = 3$; offset in block $12\,800 - 3 \times 4096 = 512$.
16. Record 40 occupies bytes $40 \times 250 = 10\,000$ to $10\,249$. $\lfloor 10\,000/1024 \rfloor = 9$ but $\lfloor 10\,249/1024 \rfloor = 10$: the record crosses a block boundary, so reading it needs two blocks.

14 Midterm Review: Weeks 1--7

The midterm exam in Week 8 covers everything from Week 1 to this week. Use this list as a checklist; each item appeared in the slides and the course notes of the week shown.

Week 1: History What an OS does; batch systems, multiprogramming, time-sharing; why idle CPUs drove OS design; UNIX (1969) and Linux (1991).

Week 2: OS basics Definitions and the kernel; extended machine vs. resource manager; goals; interrupts; storage hierarchy; dual mode; system calls (`strace`); simple, monolithic, layered, microkernel, modular, hybrid structures.

Week 3: Processes OS services; types of system calls; the process and its memory layout; process states; PCB; context switch; `fork()`.

Week 4: Threads, CPU Threads vs. processes; user and kernel threads; multithreading models; CPU–I/O bursts; scheduling criteria (utilisation, throughput, turnaround, waiting and response time).

Week 5: Scheduling, sync FCFS, SJF/SRTF, Priority, Round Robin with Gantt charts and average waiting time; race conditions; the critical-section problem; mutex locks; semaphores.

Week 6: Memory Address binding; logical vs. physical addresses; MMU; swapping; contiguous allocation; internal and external fragmentation; paging and address translation; TLB and EAT; segmentation.

Week 7: VM, files Demand paging; page faults; EAT with page faults; copy-on-write; FIFO, OPT, LRU tables; Belady's anomaly; clock; thrashing and working set; file attributes and operations; access methods with block arithmetic; directory structures; hard and symbolic links.

Numerical question types to practise: Gantt charts and waiting times (Week 5); address translation and EAT with a TLB (Week 6); EAT with page faults and page-replacement frame tables (Week 7); direct-access block arithmetic (Week 7). The best practice material is the set of review questions with worked answers at the end of each week's notes.

15 Looking Ahead: Week 8 and Beyond

- **Week 8: Midterm exam** on Weeks 1–7. Bring your student ID and pens; the exact time, room and rules (for example about calculators) will be announced on the course Telegram channel.
- **Week 9** starts the hands-on half of the course: the Linux shell and navigating the directory tree we drew this week (`pwd`, `cd`, `ls`, absolute and relative paths).
- **Week 10:** file operations (`cp`, `mv`, `rm`, `ln`, `touch`); **Week 11:** permissions (the `-rw-rw-r--` string in the inode).

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapter 10 (Virtual Memory), Chapter 13 (File-System Interface) and Section 14.1 (File-System Structure).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 21, "Beyond Physical Memory: Mechanisms": `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys.pdf`
  - Chapter 22, "Beyond Physical Memory: Policies": `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf`
  - Chapter 39, "Interlude: Files and Directories": `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- L. A. Bélády, R. A. Nelson and G. S. Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine", *Communications of the ACM* 12(6), 1969, pp. 349–353.